

Class XI | Unit 6 | weight 15%

Arrays, strings and structures

Board weight about 15% | 25 theory periods and 25 lab periods

An array stores many values of one type under one name. A string stores text. A structure stores related values of different types as one record. Most of the Class XI practical journal lives in this unit.

What you should be able to do

- Define an array, its name, size and index, and initialise arrays of different types.
- Access an element, traverse an array, and use sizeof.
- Declare, initialise and process a two-dimensional array.
- Use C strings with the terminator, and use strlen, strcat, strcpy, strcmp and substr.
- Declare a structure, create a variable, and read and display its members.

Path through the lecture

- 1 Draw five boxes with indexes 0 to 4 before any syntax.
- 2 Traverse, search, sort and average on the same small list so the loops look familiar.
- 3 Treat a string as an array of char with a hidden zero at the end.
- 4 Close with one employee record. That is the bridge to classes in Class XII.

6.1 Arrays

Array. A collection of elements of the same data type, stored in consecutive memory locations and referred to by one name and an index.

The **name** is the identifier, such as marks. The **size** is how many elements were reserved. The **index** is the position of one element. In C++ the first index is 0. An array of 5 elements uses indexes 0, 1, 2, 3 and 4. There is no element at index 5. Using one is a serious error.

You can initialise an array where you declare it. `int marks[5] = {10, 20, 30, 40, 50};` sets every element. `char grades[3] = {'A', 'B', 'C'};` is an array of characters, which is not yet a C string because there is no terminator. Access one element with the index in brackets: `marks[2]` is 30.

`sizeof(marks)` gives the storage of the whole array in bytes. `sizeof(marks[0])` gives the storage of one element. Their quotient is the number of elements. On a typical PC an int is 4 bytes, so five ints occupy 20 bytes.

Program 6.1 Initialise, access, traverse and measure an array

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {10, 20, 30, 40, 50};
    cout << "Element at index 2 = " << marks[2] << endl;
    cout << "Bytes = " << sizeof(marks) << endl;
    cout << "Elements = " << sizeof(marks) / sizeof(marks[0]) << endl;
    cout << "Traverse: ";
    for (int i = 0; i < 5; i++)
        cout << marks[i] << " ";
    cout << endl;
    return 0;
}
```

Traversal means visiting every element once. The for-loop from 0 to size-1 is the pattern you will reuse for search, total and display.

6.2 Two-dimensional arrays

Two-dimensional array. An array whose elements are arranged in rows and columns. An element needs a row index and a column index.

`int table[2][3]` has 2 rows and 3 columns, so 6 ints. In memory the rows are stored one after another. The initialiser lists the rows in order. `table[1][2]` is row 1, column 2, counting from zero, which is the last element of the second row.

Program 6.2 A 2 by 3 array

```
#include <iostream>
using namespace std;

int main() {
    int table[2][3] = {{1, 2, 3}, {4, 5, 6}};
    for (int r = 0; r < 2; r++) {
        for (int c = 0; c < 3; c++)
            cout << table[r][c] << " ";
        cout << endl;
    }
    cout << "Row 1, column 2 = " << table[1][2] << endl;
    return 0;
}
```

Addition of two matrices is element by element: each sum element is $a[i][j] + b[i][j]$. The matrices must have the same number of rows and the same number of columns. Multiplication of a 3 by 3 matrix needs three nested loops. The element in row i and column j of the product is the sum of $a[i][k] * b[k][j]$ as k runs across the row and down the column.

Program 6.3 Add and multiply 3 by 3 matrices

```
#include <iostream>
using namespace std;

int main() {
    int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int b[3][3] = {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    int sum[3][3], product[3][3];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            sum[i][j] = a[i][j] + b[i][j];
    }
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            product[i][j] = 0;
            for (int k = 0; k < 3; k++)
                product[i][j] += a[i][k] * b[k][j];
        }
    }
    cout << "Sum row0: " << sum[0][0] << " " << sum[0][1]
        << " " << sum[0][2] << endl;
    cout << "Product row0: " << product[0][0] << " "
        << product[0][1] << " " << product[0][2] << endl;
    cout << "Product row2: " << product[2][0] << " "
        << product[2][1] << " " << product[2][2] << endl;
    return 0;
}
```

In Program 6.3 the second matrix is an identity matrix, so the product equals the first matrix. The first row of the sum is 2, 2, 3 because the identity adds 1 only to the diagonal.

6.3 Searching, sorting and the average

A linear search looks at each element until it finds the key or reaches the end. Its place in the answer is the index, or a clear 'not found'. Bubble sort repeatedly walks the list and swaps a pair that is out of order. After each outer pass, one more value has settled at the end. For names stored as C strings, the comparison is `strcmp`, and the swap copies whole strings with `strcpy`.

Program 6.4 Bubble sort, name sort and linear search

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    int a[] = {5, 1, 4, 2};
    int n = 4;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
    cout << "Sorted: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;

    char names[3][10] = {"Sara", "Ali", "Hina"};
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (strcmp(names[j], names[j + 1]) > 0) {
                char temp[10];
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }
    cout << "Names: ";
    for (int i = 0; i < 3; i++)
        cout << names[i] << " ";
    cout << endl;

    int key;
    cout << "Search for: ";
    cin >> key;
    int index = -1;
    for (int i = 0; i < n; i++) {
        if (a[i] == key)
            index = i;
    }
    if (index >= 0)
        cout << "Found at index " << index << endl;
    else
        cout << "Not found" << endl;
    return 0;
}
```

Program 6.5 Average of a list

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "How many numbers? ";
    cin >> n;
    int sum = 0;
    for (int i = 0; i < n; i++) {
        int value;
        cin >> value;
        sum += value;
    }
    cout << "Average = " << sum / n << endl;
    return 0;
}
```

Program 6.5 uses integer division because the total and the count are ints. The average of 10, 20, 30 and 40 is 25 exactly. If you need a fractional average, store the total in a double.

6.4 Strings

C string. An array of char that ends with the null character `\0`. The null character is the terminator. It marks the end and is not part of the text.

`char name[20] = "Ali";` stores A, l, i, and then `\0`. The size 20 is the room available, not the length of the text. `strlen(name)` counts characters before the terminator, so the length is 3, not 20 and not 4.

String operations in this unit

Operation	What it does	Care
<code>strlen(s)</code>	Counts characters before the terminator	The terminator is not counted
<code>strcpy(dest, src)</code>	Copies src into dest, including the terminator	dest must be large enough
<code>strcat(dest, src)</code>	Joins src onto the end of dest	dest must have room for both strings
<code>strcmp(a, b)</code>	Compares a and b. The result is 0 when they are equal	A negative result means a comes first
<code>s.substr(pos, n)</code>	From a <code>std::string</code> , takes n characters starting at pos	This is a member of the string class, not a C-string function

The curriculum name `substr` belongs to the C++ string class. The C functions `strlen`, `strcpy`, `strcat` and `strcmp` belong to `cstring` and work on char arrays. Keep the two styles apart. Do not pass a `std::string` to `strcpy`.

Program 6.6 Length, reverse, join, compare and substr

```

#include <iostream>
#include <cstring>
#include <string>
using namespace std;

int main() {
    char name[40];
    cout << "Enter a single-word name: ";
    cin >> name;
    cout << "Length = " << strlen(name) << endl;
    cout << "Reverse = ";
    for (int i = strlen(name) - 1; i >= 0; i--)
        cout << name[i];
    cout << endl;

    char first[20] = "BIEK";
    char second[20] = "Karachi";
    char joined[40];
    strcpy(joined, first);
    strcat(joined, " ");
    strcat(joined, second);
    cout << "Joined = " << joined << endl;
    if (strcmp(first, second) < 0)
        cout << "BIEK comes before Karachi" << endl;

    string word = "Computer";
    cout << "Part = " << word.substr(0, 3) << endl;
    return 0;
}

```

Reversing a name is a loop from the last character down to index 0. For Ayesha the length is 6 and the reverse is ahseyA. strcpy followed by two strcat calls builds BIEK Karachi. substr(0, 3) on Computer gives Com.

6.5 Structures

Structure. A user-defined type that groups members of different types under one name. Each member has its own name.

The struct block declares the type. It does not by itself create a variable. `Employee emp;` creates a variable. Members are reached with a dot: `emp.salary`. A structure can be initialised with a brace list in the same order as the members. An array of structures is a table of records, which is the idea you will meet again in files and in databases.

Program 6.7 An employee record

```
#include <iostream>
#include <cstring>
using namespace std;

struct Employee {
    char name[30];
    char designation[20];
    int salary;
};

int main() {
    Employee emp;
    cout << "Enter name, designation and salary: ";
    cin >> emp.name >> emp.designation >> emp.salary;
    cout << "Name: " << emp.name << endl;
    cout << "Designation: " << emp.designation << endl;
    cout << "Salary: " << emp.salary << endl;
    return 0;
}
```

This version reads a single-word name with `cin >>`. A name with a space needs `getline`, and mixing `getline` with `>>` needs care about the leftover newline. For the board, a single-word name is the expected program unless the question asks for full names.

Exam point

When a 3-mark question says 'declare a structure for a book', write the struct with the members they name, then declare one variable, then show one member being assigned. Three short pieces of code answer the question.

Key terms

Term	Meaning to write in an answer
Index	The position of an element. The first index in C++ is 0.
Traversal	Visiting every element of an array once.
Null character	The '\0' terminator that ends a C string.
strlen	Returns the number of characters before the terminator.
strcmp	Compares two C strings. Zero means they are equal.
Structure	A group of related members that may have different types.
Member	One named field inside a structure.

Bring it together

- An array is one name, one type, many elements, indexes from 0.
- `sizeof` gives bytes. Divide by the size of one element to get the count.
- Two indexes select a row and a column. Matrix addition matches positions. Multiplication combines a row with a column.
- A C string ends with `\0`. `strlen`, `strcpy`, `strcat` and `strcmp` are the C functions. `substr` belongs to `std::string`.
- A structure groups different types. The dot operator reaches a member.

Do this in the practical

- 1 Sort a numeric list and a short list of names. Search for a value and print its index.
- 2 Average a list. Add and multiply two 3 by 3 matrices.
- 3 Display a name in reverse using `strlen`.

- 4 Create an Employee structure with name, designation and salary, then display one employee.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. The index of the first element of a C++ array is:

- A. 1
- B. 0
- C. -1
- D. The size of the array

2. `int a[4]` reserves indexes:

- A. 1 to 4
- B. 0 to 3
- C. 0 to 4
- D. 4 to 8

3. `strlen` on a C string counts:

- A. The array capacity, including unused room
- B. Characters before the terminator
- C. Only vowels
- D. The number of arrays in the program

4. `strcmp` returns 0 when:

- A. The strings differ
- B. The strings are equal
- C. The first string is empty always
- D. A file is closed

5. `strcpy(dest, src)` copies:

- A. `src` into `dest`
- B. `dest` into `src`
- C. Only the first character, and stops
- D. A structure member into ROM

6. A structure is used when:

- A. All values must be the same type and nothing else is allowed
- B. Related members of different types form one record
- C. Only one `int` is needed
- D. A bus topology is drawn

7. `table[1][2]` in an array declared `[2][3]` is:

- A. Always out of range
- B. The last column of the second row
- C. The first element
- D. A string function

8. The member operator for a structure variable is:

- A. `->`
- B. `.`
- C. `::`
- D. `&`

Answer key

Q	Answer	Why this is the answer
1	B	C++ arrays are numbered from zero.
2	B	Four elements use indexes 0, 1, 2 and 3.
3	B	The terminator marks the end and is not included in the length.
4	B	A zero result means the two C strings match.
5	A	The destination is the first argument.
6	B	Structures group different types. Arrays group one type.
7	B	Row index 1 is the second row. Column index 2 is the third column.
8	B	A structure variable uses the dot. The arrow is used with a pointer to a structure.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define an array. What are its name, size and index?

Model answer

An array is a collection of elements of the same type in consecutive memory, addressed by one name and an index. The name identifies the collection. The size is the number of elements reserved. The index selects one element. In C++ the first index is 0.

Q2. What does sizeof tell you about an array?

Model answer

sizeof(array) is the number of bytes occupied by the whole array. Dividing by sizeof(array[0]) gives the number of elements. It does not return the length of a C string; strlen does that.

Q3. Why does a C string need '\0'?

Model answer

The terminator marks where the text ends inside a char array that may be larger than the text. strlen, strcpy and the output of cout stop at that character. Without it, a string function reads on into neighbouring memory.

Q4. Differentiate an array and a structure.

Model answer

An array holds many elements of one type, selected by an index. A structure holds named members that may have different types, selected by the member name. A class register of marks can be an array. One student's name, class and fee can be a structure.

Q5. Write the statement that joins two C strings.

Model answer

Place the first string in a char array large enough for both, then call strcat(first, second). strcat appends the second string and its terminator onto the first. strcpy is the copy, not the join.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain one-dimensional and two-dimensional arrays, with initialisation and access.

- 1 Define an array and the terms name, size and index. State that the first index is 0.
- 2 Show a declaration with an initialiser and one access, such as marks[2].
- 3 Show a traversal loop and the sizeof quotient.
- 4 Define a two-dimensional array as rows and columns. Show int t[2][3] and the element t[1][2].
- 5 State the rule for matrix addition and the three-loop pattern for multiplication.

Question 2. Explain C strings and write the purpose of strlen, strcpy, strcat, strcmp and substr.

- 1 Define a C string as a char array ending in \0. Distinguish capacity from length.
- 2 Give strlen, strcpy, strcat and strcmp with one sentence and one caution each.
- 3 Show substr on a std::string and say it is not a cstring function.
- 4 Describe the reverse-name loop from length-1 down to 0.